

Python podstawy, zalety i wady

1. Wstęp teoretyczny

Python to wysokopoziomowy, ogólnego przeznaczenia język programowania. Został stworzony z naciskiem na czytelność, prostotę i szybkość tworzenia kodu.

Python jest:

- interpretowany - kod jest wykonywany przez interpreter,
- dynamicznie typowany - typ zmiennej jest ustalany podczas działania programu,
- wieloplatformowy - działa na Windows, Linuxie i macOS,
- ogólnego przeznaczenia - można wykorzystać go w wielu różnych dziedzinach.

Python wykorzystuje się między innymi do:

- tworzenia aplikacji,
 - automatyzacji zadań,
 - tworzenia stron internetowych,
 - analizy danych,
 - sztucznej inteligencji i uczenia maszynowego.
-

2. Podstawowe elementy Pythona

Zmienne

Zmienna służy do przechowywania danych.

```
name = "Jan"
```

```
age = 17
```

```
height = 1.80
```

```
student = True
```

Python sam rozpoznaje typ przechowywanej wartości.

Pobieranie danych

Do pobierania danych od użytkownika służy `input()`.

```
name = input("Podaj imię: ")
```

```
print(name)
```

`input()` zwraca dane jako tekst. Jeśli potrzebujemy liczby, możemy użyć np. `int()`:

```
age = int(input("Podaj wiek: "))
```

`int()` zamienia wartość na liczbę całkowitą.

3. Wyświetlanie danych

Do wyświetlania informacji służy `print()`.

```
name = "Jan"
```

```
age = 17
```

```
print(name)
```

```
print(age)
```

Formatowanie

Bardzo wygodnym sposobem tworzenia tekstu zawierającego wartości zmiennych.

Przed tekstem umieszczamy literę `f`, a zmienne wpisujemy w `{}`:

```
name = "Jan"
```

```
age = 17
```

```
print(f"Mam na imię {name} i mam {age} lat.")
```

Formatowanie liczb

pozwała również kontrolować sposób wyświetlania liczb.

Dwa miejsca po przecinku

`:.2f` oznacza wyświetlenie liczby z dokładnością do dwóch miejsc po przecinku.

```
price = 12.34567
```

```
print(f"Cena: {price:.2f} zł")
```

Wynik:

Cena: 12.35 zł

Separowanie dużych liczb

Możemy użyć `,:`:

```
number = 1000000
```

```
print(f"Liczba: {number:,}")
```

Wynik:

Liczba: 1,000,000

4. Instrukcje warunkowe

Do podejmowania decyzji wykorzystujemy `if`, `elif` oraz `else`.

```
age = 18
```

```
if age >= 18:
```

```
    print("Pełnoletni")
```

```
else:
```

```
    print("Niepełnoletni")
```

- if - oznacza "jeżeli"
- elif - oznacza "w przeciwnym razie, jeżeli"
- else - oznacza "w przeciwnym przypadku"

Wcięcia

W Pythonie wcięcia są bardzo ważne. Określają, które instrukcje należą do danego bloku kodu.

```
if age >= 18:
```

```
    print("Pełnoletni")
```

5. Pętle

Pętle służą do wielokrotnego wykonywania kodu.

Przykład pętli for:

```
for i in range(5):
```

```
    print(i)
```

6. Listy

Lista pozwala przechowywać wiele wartości w jednej zmiennej.

```
grades = [4, 5, 3, 5, 4]
```

Elementy listy mają swoje indeksy. Indeksy zaczynają się od 0.

7. Zasady dobrego pisania kodu

Podczas programowania warto pamiętać o kilku podstawowych zasadach:

- stosuj czytelne nazwy zmiennych,
- dbaj o prawidłowe wcięcia,
- nie powtarzaj niepotrzebnie tego samego kodu,
- pisz kod w prosty i uporządkowany sposób,

- stosuj komentarze przy bardziej skomplikowanych fragmentach,
 - sprawdzaj poprawność danych wprowadzanych przez użytkownika,
 - dbaj o to, aby kod był łatwy do późniejszej modyfikacji.
-

8. Zalety Pythona

1. Czytelność

Python posiada prostą składnię zbliżoną do języka naturalnego. Wcięcia sprawiają, że kod jest uporządkowany i łatwy do czytania.

2. Szybkość tworzenia programów

W Pythonie zazwyczaj trzeba napisać mniej kodu niż np. w Javie czy C#. Nie trzeba stosować wielu elementów składniowych, takich jak średniki czy rozbudowane deklaracje typów.

3. Duża liczba bibliotek

Python posiada ogromny ekosystem gotowych bibliotek.

Przykłady:

- NumPy - obliczenia matematyczne,
- Pandas - analiza danych,
- PyTorch - sztuczna inteligencja i uczenie maszynowe,
- Django i Flask - tworzenie aplikacji internetowych.

4. Wieloplatformowość

Programy napisane w Pythonie mogą działać na różnych systemach operacyjnych, m.in. Windows, Linux i macOS.

5. Wszechstronność

Python znajduje zastosowanie w wielu dziedzinach, np. automatyzacji, tworzeniu stron, analizie danych, AI, nauce i tworzeniu skryptów.

6. Duża społeczność

Duża liczba użytkowników sprawia, że łatwo znaleźć dokumentację, tutoriale i rozwiązania typowych problemów.

9. Wady Pythona

1. Mniejsza wydajność

Python jest zazwyczaj wolniejszy od języków takich jak C++, C# czy Java w zadaniach wymagających dużej liczby obliczeń.

2. Typowanie dynamiczne

Typ zmiennej jest ustalany podczas działania programu.

```
number = 10
```

```
number = "tekst"
```

Jest to wygodne, ale niektóre błędy mogą zostać wykryte dopiero podczas uruchamiania programu.

3. Zarządzanie zależnościami

W większych projektach trzeba kontrolować wersje bibliotek i zależności. Wykorzystuje się do tego m.in. środowiska wirtualne, np. venv.

4. Słabe zastosowanie w aplikacjach mobilnych

Python nie jest popularnym wyborem do tworzenia aplikacji mobilnych. W tej dziedzinie częściej wykorzystuje się technologie przeznaczone specjalnie dla Androida i iOS.